UNIVERSIDADE FEDERAL DO CEARÁ

TECHNICAL REPORT

Aluno: Maverick Alekyne de Sousa Ribeiro

1. Introdução

*O dataset utilizado nesta análise é intitulado **Brazilian Headlines Sentiments**, obtido no Kaggle. Este conjunto de dados oferece uma rica base para a análise de sentimentos e mineração de texto, com informações provenientes de diversas fontes jornalísticas brasileiras.*

Descrição do Dataset

O dataset contém um total de 15 colunas, com exemplos representando manchetes e seus correspondentes em português e inglês, além de informações detalhadas sobre sentimentos associados às manchetes. As colunas principais incluem:

- ***Unnamed: 0**: Índice dos registros.*
- ***website**: Nome da fonte jornalística.*
- ***keywords**: Palavras-chave associadas à manchete.*
- ***headlinePortuguese**: Manchete original em português.*
- ***headlineEnglish**: Tradução da manchete para o inglês.*
- ***sentimentScorePortuguese / sentimentScoreEnglish**: Pontuação de sentimento em português e inglês (valores contínuos que indicam a polaridade do sentimento: negativo, neutro ou positivo).*
- ***sentimentMagnitudePortuguese / sentimentMagnitudeEnglish**: Intensidade do sentimento, ou seja, a força com que a polaridade do sentimento se manifesta.*
- ***isOnline**: Indicador booleano sobre se o conteúdo está online ou não.*
- ***onlineStartDate / onlineEndDate**: Data de início e fim da disponibilidade online.*
- ***onlineTotalTimeMS**: Tempo total online em milissegundos.*

Contexto e Aplicações

Este dataset é uma ferramenta valiosa para:

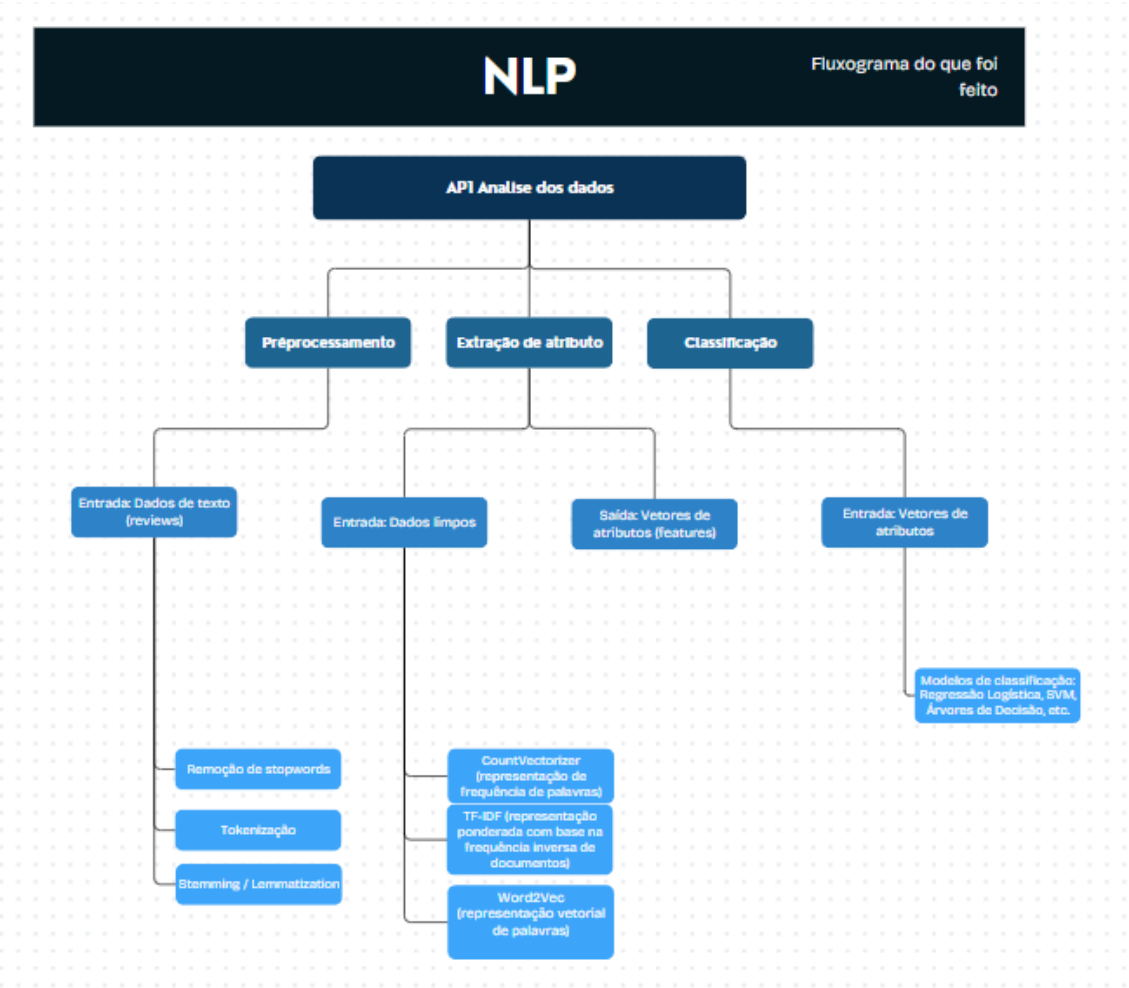
1. ***Análise de Sentimentos**: Explorar polaridades e magnitudes de sentimentos em textos jornalísticos em dois idiomas.*
2. ***Mineração de Texto**: Extração de palavras-chave e padrões em manchetes.*

Que são as duas atividades que vão ser usadas para realizar essa prova.

UNIVERSIDADE FEDERAL DO CEARÁ

O dataset representa um cenário real do ambiente midiático brasileiro, abordando tópicos variados, como política, economia, sociedade, e figuras públicas. Manchetes relacionadas a personalidades como ***Bolsonaro****,* ***Lula****,* ***Moro*** *e* ***Weintraub*** *são frequentes, proporcionando uma base sólida para análises voltadas a eventos específicos ou tendências mais amplas.*

Fluxograma



UNIVERSIDADE FEDERAL DO CEARÁ

Análise exploratória dos dados

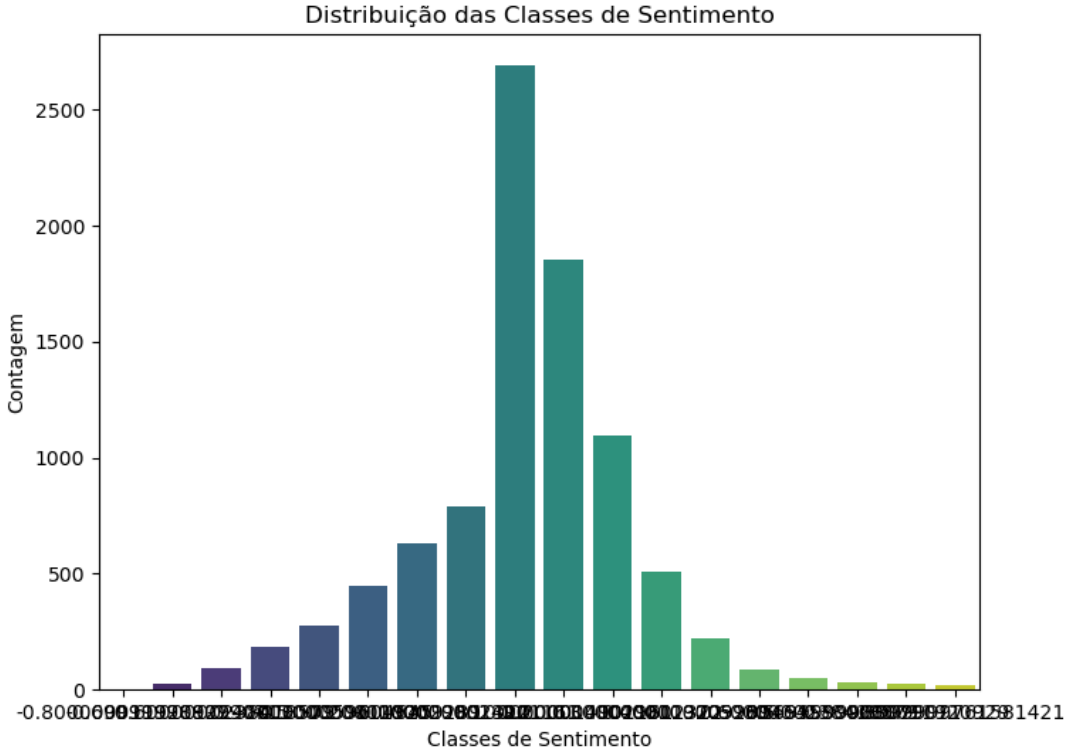


Gráfico 1: Distribuição das Classes de Sentimento

O gráfico apresenta a **distribuição das classes de sentimento**. Nesse contexto, cada classe corresponde a um intervalo contínuo, relacionado a uma escala de sentimentos sendo o score de análise de sentimentos. Observa-se uma distribuição assimétrica com concentração maior na faixa central, sugerindo que a maior parte dos valores se encontra próxima a um sentimento neutro ou levemente positivo. As extremidades negativas e positivas possuem menos ocorrências, indicando que sentimentos extremos são menos comuns no conjunto de dados analisados. Essa distribuição reflete uma variabilidade mais acentuada nos sentimentos intermediários. Sendo esse gráfico antes da distribuição de faixas nos dados.

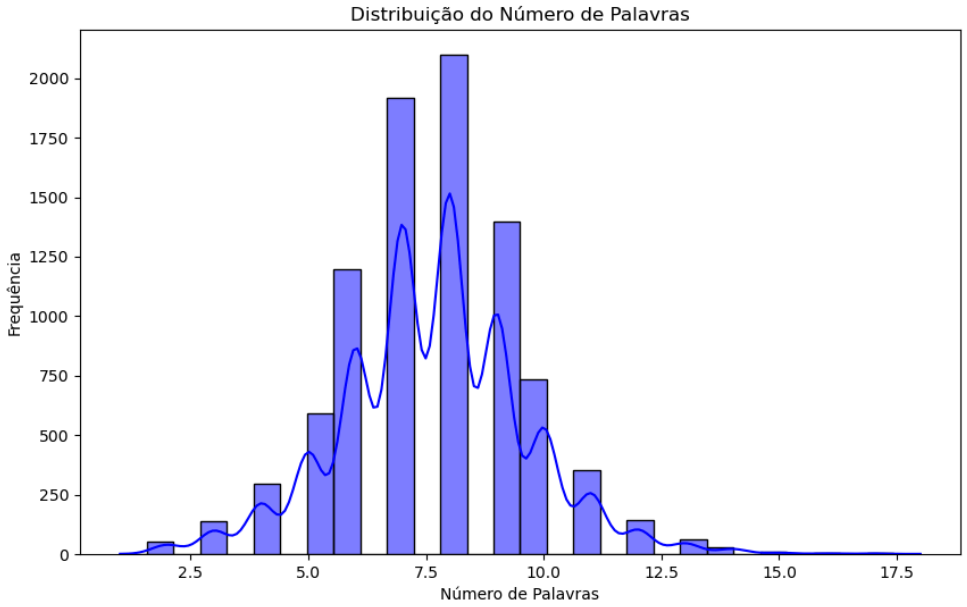


Gráfico 2: Distribuição do Número de Palavras

*O gráfico mostra a **distribuição do número de palavras** em uma amostra de textos. A curva sugere uma distribuição aproximadamente normal, com uma moda (pico) em torno de 9 palavras por texto. Isso indica que a maioria dos textos na base de dados contém um número moderado de palavras, enquanto textos muito curtos ou muito longos são menos frequentes. A sobreposição de um histograma e de uma curva suavizada permite visualizar tanto a frequência exata quanto a tendência geral dessa distribuição, facilitando a análise do comportamento textual no corpus estudado.*

Análise da Coluna headlinePortuguese

- ***Número de textos:*** *A coluna de títulos contém 9027 entradas.*
- ***Comprimento médio das palavras:*** *Cada texto apresenta uma média de 7,65 palavras.*
- ***Comprimento médio dos caracteres:*** *Em média, os textos têm 46,64 caracteres.*
- ***Texto mais longo (palavras):*** *O título mais longo contém 18 palavras.*
- ***Texto mais longo (caracteres):*** *O título mais longo possui 106 caracteres.*

UNIVERSIDADE FEDERAL DO CEARÁ

Análise da Coluna sentimentScorePortuguese

- *A pontuação de sentimento varia de -0.8 a 0.9, com a maioria das pontuações concentradas perto do zero, indicando uma predominância de sentimentos neutros ou levemente positivos/negativos.*
- *A distribuição das classes de sentimento revelou que as pontuações mais comuns são 0.0 (2692 ocorrências), seguidas de 0.1 (1852 ocorrências) e 0.2 (1093 ocorrências), com pontuações negativas como -0.1 e -0.2 também frequentes. Pontuações extremas, como -0.8 e 0.9, aparecem muito raramente.*

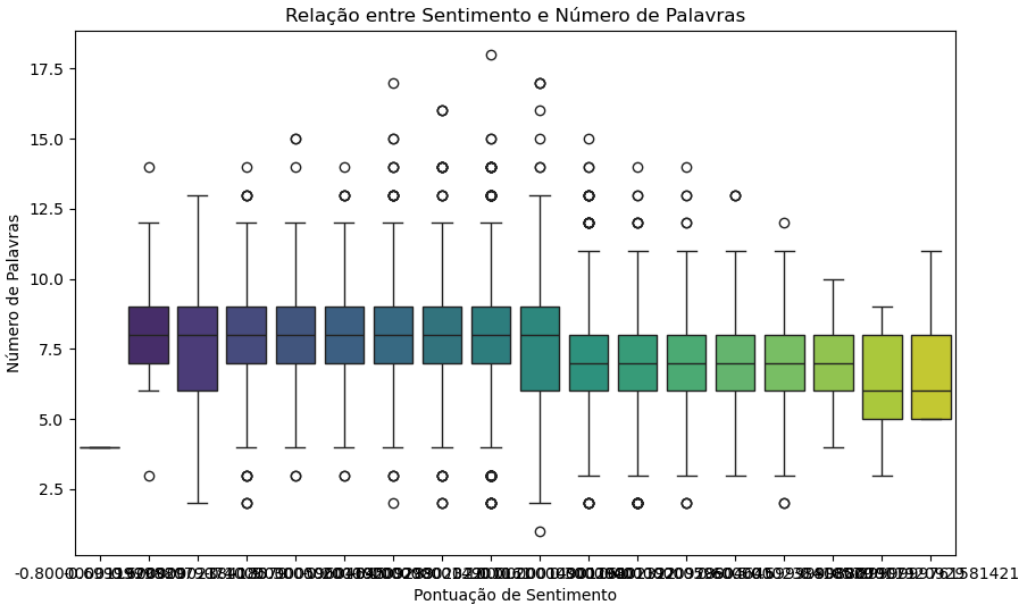


Gráfico 3: Relação entre Sentimento e Número de Palavras

O primeiro gráfico é um boxplot que apresenta a ***relação entre a pontuação de sentimento e o número de palavras*** *nos textos. Cada caixa representa a distribuição do número de palavras para um intervalo específico de pontuação de sentimento. Observa-se que, de forma geral, o número de palavras não varia significativamente em função da pontuação de sentimento, mantendo uma mediana estável em torno de 9 palavras. Contudo, há algumas dispersões nas extremidades dos sentimentos, evidenciadas pelos outliers. sugerindo que, enquanto a maioria dos textos tem comprimento similar, os textos com sentimentos mais extremos podem ocasionalmente apresentar variações maiores no número de palavras.*

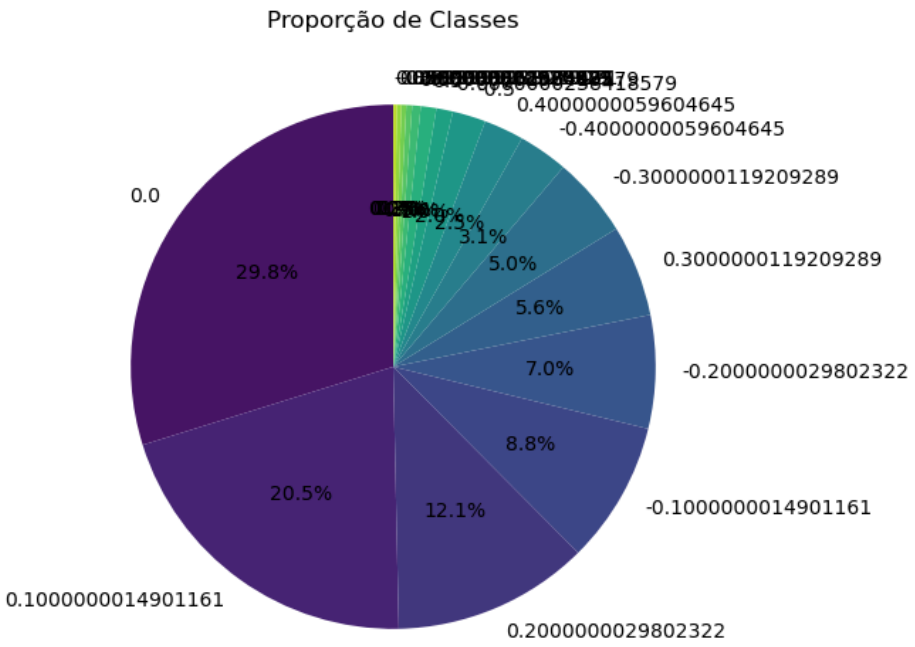


Gráfico 4: Proporção de Classes

O segundo gráfico é um gráfico de pizza que exibe a ***proporção de classes de sentimento*** *no conjunto de dados. A maior parcela corresponde a uma classe específica (29,8%), seguida por outra com 20,5%. As demais classes apresentam proporções decrescentes, indicando uma concentração maior em poucas categorias principais, enquanto as restantes possuem representatividade menor. Essa visualização destaca o desequilíbrio entre as classes, o que pode impactar análises ou modelos preditivos que dependam desses dados, demandando estratégias específicas de balanceamento.*

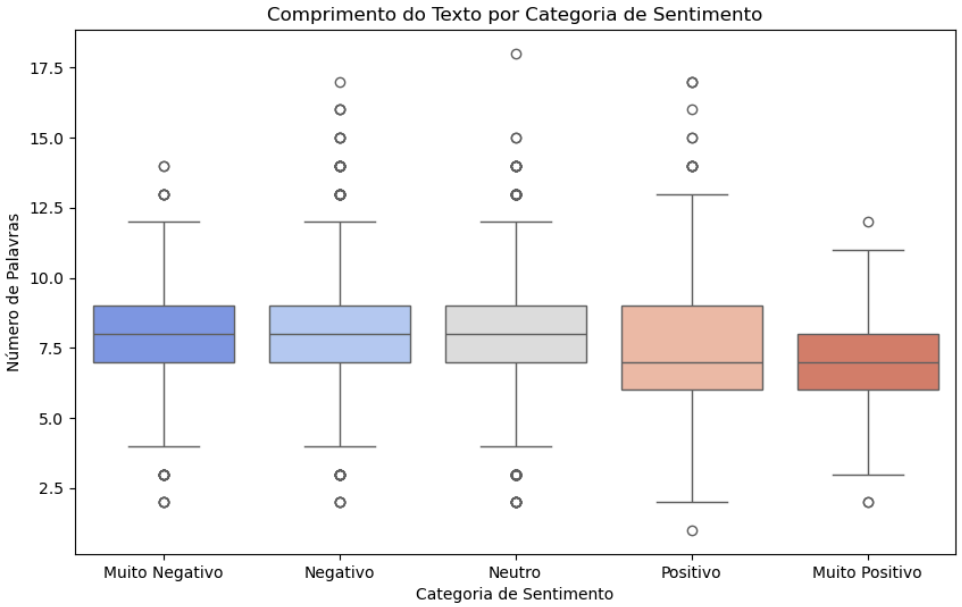


Gráfico 5: Comprimento do Texto por Categoria de Sentimento

O gráfico é um boxplot que apresenta a ***distribuição do comprimento dos textos (número de palavras)*** *por categorias de sentimento, classificadas como "Muito Negativo", "Negativo", "Neutro", "Positivo" e "Muito Positivo". As medianas de todas as categorias são similares, com valores próximos a 9 palavras. A distribuição do número de palavras em cada categoria é relativamente consistente, mas observa-se uma maior dispersão (outliers) em categorias extremas como "Muito Negativo" e "Muito Positivo". Isso sugere que textos mais longos ou mais curtos podem ocorrer com maior frequência em extremos emocionais, mas a maioria dos textos mantém um comprimento uniforme.*

Distribuição por Faixas de Sentimento

A análise mostrou que as faixas de sentimento predominantes são as de sentimentos neutros (0.0) e ligeiramente positivos (0.1 e 0.2), enquanto faixas mais extremas (como 0.9 e -0.8) apresentam um número muito reduzido de ocorrências. As classes raras, com menos de 50 ocorrências, incluem sentimentos intensamente positivos ou negativos, como 0.9, -0.8, -0.7 e 0.6.

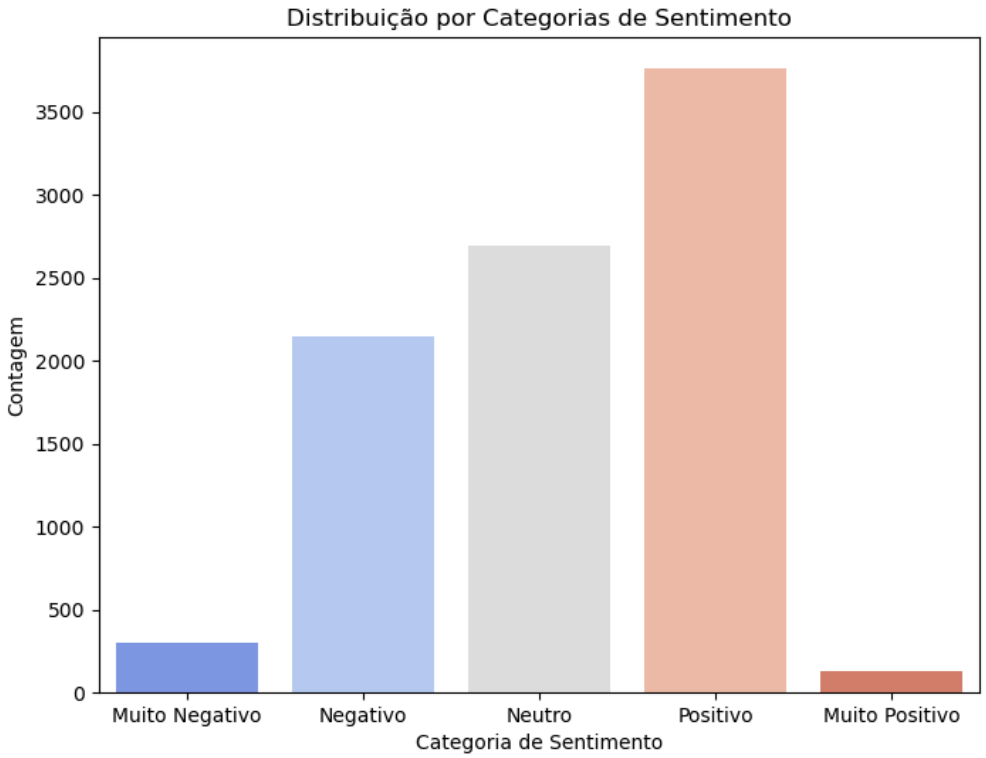


Gráfico 6: Distribuição por Categorias de Sentimento

Análise das Classes de Sentimento

Foi realizada a categorização dos dados em cinco faixas de sentimento:

- ***Positivo (3763):*** *A maior parte dos textos está associada a sentimentos positivos, com a pontuação geralmente em torno de 0.1.*
- ***Neutro (2692):*** *A segunda maior classe, refletindo uma pontuação de sentimento muito próxima de zero.*
- ***Negativo (2143):*** *Os textos com sentimentos negativos, com pontuações variando entre -0.1 e -0.2.*
- ***Muito Negativo (303):*** *Classe com textos que apresentam sentimentos fortemente negativos, com pontuações abaixo de -0.5.*
- ***Muito Positivo (126):*** *Classe menos expressiva, com textos que apresentam pontuações de sentimento acima de 0.5.*

UNIVERSIDADE FEDERAL DO CEARÁ

Questão 1-

Pré-processamento de Texto: Preparando Dados para Análise

O pré-processamento de dados textuais é uma etapa fundamental no pipeline de análise de dados, especialmente quando lidamos com tarefas de processamento de linguagem natural (NLP). Essa fase visa transformar os dados brutos em uma forma mais adequada para modelos de aprendizado de máquina, assegurando que as informações sejam mais precisas, limpas e consistentes. O código aqui apresentado foi desenvolvido com o objetivo de realizar uma série de transformações essenciais para a preparação de um dataset textual, permitindo que ele seja utilizado de forma eficiente em tarefas subsequentes de análise de sentimentos, classificação, extração de atributos e muito mais.

Objetivos do Pré-processamento

O principal objetivo do pré-processamento de texto é transformar os dados textuais em um formato que seja adequado para modelos de aprendizado de máquina e outras técnicas de análise. Entre os principais objetivos desta fase, destacam-se:

- ***Remoção de ruídos**: Eliminar elementos indesejados, como pontuações, menções de usuários e palavras comuns (stopwords), que não agregam valor ao processo analítico.*
- ***Padronização linguística**: Garantir que todas as palavras estejam em um formato uniforme, por meio da conversão para letras minúsculas, remoção de inflexões e variações gramaticais.*
- ***Redução da dimensionalidade**: Agrupar variações linguísticas, como palavras derivadas, em uma forma única para reduzir a complexidade do modelo.*

Etapas do Pré-processamento

A implementação apresentada aqui segue uma abordagem modular e flexível, dividindo o pré-processamento em várias etapas que podem ser facilmente ajustadas ou estendidas conforme necessário.

1. Remoção de Pontuação

Uma das primeiras transformações realizadas é a remoção de pontuações. Este processo elimina caracteres como vírgulas, pontos, pontos de interrogação, entre outros, que muitas vezes não têm relevância para a análise.

2. Conversão para Minúsculas

A conversão do texto para letras minúsculas é uma etapa crucial para garantir a uniformidade. Palavras como "Python" e "python" seriam tratadas de forma distinta sem essa conversão.

3. Remoção de Menções

Em dados provenientes de redes sociais ou plataformas como Twitter, é comum encontrar menções a usuários (ex: @usuario). Estas menções são removidas, pois geralmente não agregam informações úteis à análise de sentimentos ou classificação de texto.

4. Remoção de Stopwords

As stopwords são palavras comuns que geralmente não têm valor semântico significativo e podem ser descartadas, como preposições e artigos ("o", "a", "de", "para"). A remoção dessas palavras ajuda a reduzir o ruído e foca a análise nas palavras mais importantes.

5. Stemming

O Stemming é uma técnica que reduz palavras para suas raízes. Por exemplo, "correr" e "correndo" podem ser reduzidas a "corr". Essa técnica é útil para normalizar as variações das palavras, embora, em alguns casos, a palavra resultante possa não ser uma palavra real.

6. Lematização

A Lematização é uma abordagem mais sofisticada do que o Stemming. Ao contrário do Stemming, a lematização reduz as palavras à sua forma base e correta, garantindo que palavras como "amando" ou "correndo" sejam transformadas corretamente em "amar" e "correr".

7. Aplicação do Pré-processamento

A função preprocess_text integra todas essas etapas, permitindo que o usuário aplique o pré-processamento completo em um único texto, com a escolha entre Stemming e Lematização conforme necessário.

8. Pré-processamento em DataFrames

Além de realizar o pré-processamento em textos individuais, a função preprocess_dataframe permite que o pré-processamento seja aplicado a uma coluna específica de um DataFrame, tornando a manipulação de grandes volumes de dados muito mais simples.

Conclusão

O pré-processamento de texto é uma etapa crucial para qualquer análise que envolva dados textuais. A implementação apresentada aqui proporciona uma maneira flexível e eficiente de preparar textos para tarefas como análise de sentimentos, classificação e extração de atributos. Com a capacidade de escolher entre stemming ou lematização e a possibilidade de aplicar o pré-processamento em grandes datasets, essa estrutura torna o processo de limpeza e transformação de dados muito mais ágil e escalável.

Questão 2-

Extração de Atributos de Texto com o AttributeExtractor

Introdução

O código desenvolvido possui uma classe chamada AttributeExtractor, cuja finalidade é realizar a extração de diversos atributos a partir de textos. Essas extrações são úteis em tarefas de processamento de linguagem natural (PLN), como análise de sentimentos, classificação de texto, entre outros. A classe oferece diferentes métodos para transformar os textos em representações numéricas, como análise estatística, contagem de palavras, TF-IDF, matriz de coocorrência e Word2Vec.

Métodos de Extração

1. ***Análise Estatística***
   O método analyze_statistics calcula métricas básicas sobre os textos, como o número médio de palavras e caracteres, bem como o maior número de palavras e caracteres em um único texto. Isso pode ser útil para entender a distribuição dos textos no dataset.
   Parâmetros:
   - *df: DataFrame que contém os textos.*
2. ***Retorna:***
   - *Um dicionário com as estatísticas calculadas.*
3. ***CountVectorizer***
   O método extract_count_vectorizer usa o CountVectorizer do scikit-learn para transformar os textos em uma matriz de contagem de palavras. Este método converte os textos em uma representação esparsa, onde cada coluna corresponde a uma palavra (feature) e cada linha representa um texto.
   Parâmetros:
   - *df: DataFrame com os textos.*
   - *max_features: Número máximo de palavras a serem consideradas.*

4. ***Retorna:***
    - *Uma matriz esparsa de contagem de palavras e a lista de features (palavras).*
5. ***TF-IDF***
   O método extract_tfidf aplica a técnica de TF-IDF (Term Frequency-Inverse Document Frequency), que é uma das formas mais comuns de representar textos numericamente. O TF-IDF atribui um peso a cada palavra, que reflete sua importância no contexto do corpus. Este método também retorna uma matriz esparsa com os valores calculados e a lista de features.
   Parâmetros:
    - *df: DataFrame com os textos.*
    - *max_features: Número máximo de palavras a serem consideradas.*
6. ***Retorna:***
    - *Uma matriz esparsa de valores TF-IDF e a lista de palavras.*
7. ***Matriz de Coocorrência***
   O método extract_cooccurrence_matrix gera uma matriz de coocorrência entre as palavras do corpus. A coocorrência é útil para entender a proximidade e a associação entre diferentes termos. A matriz gerada é esparsa e simétrica, onde as linhas e colunas representam palavras, e os valores refletem a frequência com que as palavras aparecem juntas nos textos.
   Parâmetros:
    - *df: DataFrame com os textos.*
8. ***Retorna:***
    - *A matriz de coocorrência e a lista de palavras presentes no corpus.*
9. ***Word2Vec***
   O método train_word2vec usa o algoritmo Word2Vec para aprender representações vetoriais das palavras (embeddings). O modelo é treinado a partir dos textos do DataFrame, e os embeddings podem ser usados para capturar relações semânticas e contextuais entre palavras.
   Parâmetros:
    - *df: DataFrame com os textos.*
    - *vector_size: Número de dimensões dos vetores de palavras.*
    - *window: Tamanho da janela de contexto.*
    - *min_count: Frequência mínima das palavras para serem consideradas no treinamento.*
10. ***Retorna:***
    - *O modelo Word2Vec treinado.*
11. ***Obter Embedding de Palavra***
    O método get_word_embedding permite obter o vetor de embedding de uma palavra específica, que foi treinada utilizando o modelo Word2Vec. Isso pode

ser útil para realizar análises semânticas e comparar palavras entre si.
Parâmetros:
- *word: Palavra para a qual se deseja obter o embedding.*

12. ***Retorna:***
    - *O vetor de embedding da palavra.*

Análise das Estatísticas e Características dos Textos

A análise realizada sobre os textos revelou alguns resultados significativos sobre a estrutura dos dados. A média de palavras por texto foi de aproximadamente 7,65, com um número médio de caracteres por texto de 46,64. O maior número de palavras por texto foi 18, enquanto o maior número de caracteres foi 106, indicando uma variação considerável no comprimento dos textos.

Em termos de características extraídas do conjunto de dados, o ***CountVectorizer*** *gerou um conjunto de features contendo palavras e números como "04", "05", "10", "100", "1000", "1045", "106", "10ª", "11" e "112". Essas mesmas features foram também identificadas nos resultados do* ***TF-IDF****, sugerindo que a frequência das palavras no corpus não apresentou grandes variações entre as diferentes abordagens de vetorização. Além disso, a* ***Coocorrência*** *também resultou nas mesmas features, indicando que a distribuição dos termos nos textos é consistente nas diferentes representações vetoriais.*

O vocabulário gerado a partir da análise de palavras revela a presença de termos com forte conotação política, como "bolsonar", "weintraub", "stf", "govern", entre outros. Esses termos indicam um possível viés de conteúdo relacionado a questões políticas e governamentais.

Por fim, um vetor de exemplo gerado pelos métodos de vetorização mostra valores que representam a importância relativa de cada palavra para o texto analisado. O vetor, com valores tanto positivos quanto negativos, reflete as interações e a relevância das palavras em relação ao conteúdo específico do texto.

Essas observações sugerem que a análise de texto foi capaz de identificar e capturar características essenciais do conteúdo, que podem ser exploradas em modelos de aprendizado de máquina ou em estudos mais aprofundados sobre o tema dos textos analisados.

Questão 3

Resultados do Item (b): Comparação entre métodos de extração

Método Count_Vectorizer

	precision	*recall*	*f1-score*	*support*
negative	*0.80*	*0.79*	*0.80*	*1027*
positive	*0.73*	*0.74*	*0.74*	*779*
accuracy			*0.77*	*1806*
macro avg	*0.77*	*0.77*	*0.77*	*1806*
weighted avg	*0.77*	*0.77*	*0.77*	*1806*

*O modelo de classificação com **CountVectorizer** obteve os seguintes resultados:*

- ***Classe negativa**:*
  - ***Precisão**: 80%, **Recall**: 79%, **F1-score**: 80%*
  - *Bom desempenho na identificação de exemplos negativos.*
- ***Classe positiva**:*
  - ***Precisão**: 73%, **Recall**: 74%, **F1-score**: 74%*
  - *Desempenho ligeiramente inferior, com mais falsos positivos.*
- ***Métricas globais**:*
  - ***Acurácia**: 77%, **Macro avg**: 77%, **Weighted avg**: 77%*
  - *O modelo apresenta bom desempenho geral, com ligeira vantagem para a classe negativa.*

Em resumo, o modelo tem um desempenho equilibrado, com maior precisão na classe negativa e um desempenho aceitável para a classe positiva.

Método tfidf

	precision	*recall*	*f1-score*	*support*
negative	*0.79*	*0.78*	*0.79*	*1027*
positive	*0.71*	*0.74*	*0.72*	*779*
accuracy			*0.76*	*1806*
macro avg	*0.75*	*0.76*	*0.75*	*1806*
weighted avg	*0.76*	*0.76*	*0.76*	*1806*

*O modelo de classificação com **TF-IDF** obteve os seguintes resultados:*

- ***Classe negativa**:*
  - ***Precisão**: 79%, **Recall**: 78%, **F1-score**: 79%*
  - *Desempenho bom na identificação de exemplos negativos.*
- ***Classe positiva**:*
  - ***Precisão**: 71%, **Recall**: 74%, **F1-score**: 72%*
  - *Desempenho inferior em comparação com a classe negativa, com mais falsos positivos.*
- ***Métricas globais**:*
  - ***Acurácia**: 76%, **Macro avg**: 75%, **Weighted avg**: 76%*
  - *O modelo apresenta bom desempenho geral, mas com menor eficácia na classe positiva.*

Em resumo, o modelo com TF-IDF tem um desempenho aceitável, mas ainda há margem para melhorias, especialmente na classe positiva.

Método word2vec

	precision	*recall*	*f1-score*	*support*
negative	*0.67*	*0.80*	*0.73*	*1027*
positive	*0.64*	*0.48*	*0.55*	*779*
accuracy			*0.66*	*1806*
macro avg	*0.66*	*0.64*	*0.64*	*1806*
weighted avg	*0.66*	*0.66*	*0.66*	*1806*

*O modelo de classificação com **Word2Vec** obteve os seguintes resultados:*

- ***Classe negativa**:*
  - ***Precisão**: 67%, **Recall**: 80%, **F1-score**: 73%*
  - *Bom desempenho na identificação de exemplos negativos.*
- ***Classe positiva**:*
  - ***Precisão**: 64%, **Recall**: 48%, **F1-score**: 55%*
  - *Desempenho abaixo do esperado na classe positiva, com muitos falsos negativos.*
- ***Métricas globais**:*
  - ***Acurácia**: 66%, **Macro avg**: 66%, **Weighted avg**: 66%*
  - *Desempenho moderado, com a classe positiva apresentando uma maior margem para melhoria.*

Em resumo, o modelo com Word2Vec apresenta um desempenho razoável, mas precisa melhorar na classificação da classe positiva.

Resultados do Item (c): Comparação entre lematização e stemming

Pré-processamento: lemmatization

	precision	*recall*	*f1-score*	*support*
negative	*0.80*	*0.79*	*0.79*	*1027*
positive	*0.72*	*0.74*	*0.73*	*779*
accuracy			*0.76*	*1806*
macro avg	*0.75*	*0.76*	*0.76*	*1806*
weighted avg	*0.76*	*0.76*	*0.76*	*1806*

*Após aplicar o **pré-processamento com lematização**, os resultados obtidos foram:*

- ***Classe negativa**:*
  - ***Precisão**: 80%, **Recall**: 79%, **F1-score**: 79%*
  - *Bom desempenho na identificação de exemplos negativos.*
- ***Classe positiva**:*
  - ***Precisão**: 72%, **Recall**: 74%, **F1-score**: 73%*
  - *Desempenho razoável na classe positiva, com bons resultados em recall.*
- ***Métricas globais**:*
  - ***Acurácia**: 76%, **Macro avg**: 75%, **Weighted avg**: 76%*
  - *Resultados sólidos, com um equilíbrio entre precisão e recall.*

Em resumo, o modelo com lematização teve um bom desempenho geral, especialmente na classe negativa, e um desempenho razoável na classe positiva.

Pré-processamento: stemming

	precision	*recall*	*f1-score*	*support*
negative	*0.79*	*0.79*	*0.79*	*1027*
positive	*0.72*	*0.72*	*0.72*	*779*
accuracy			*0.76*	*1806*
macro avg	*0.76*	*0.76*	*0.76*	*1806*
weighted avg	*0.76*	*0.76*	*0.76*	*1806*

*Após aplicar o **pré-processamento com stemming**, os resultados obtidos foram:*

- ***Classe negativa**:*
  - ***Precisão**: 79%, **Recall**: 79%, **F1-score**: 79%*
  - *Bom desempenho na classe negativa.*
- ***Classe positiva**:*
  - ***Precisão**: 72%, **Recall**: 72%, **F1-score**: 72%*
  - *Desempenho consistente na classe positiva.*
- ***Métricas globais**:*
  - ***Acurácia**: 76%, **Macro avg**: 76%, **Weighted avg**: 76%*
  - *Resultados equilibrados entre as classes, com desempenho razoável.*

Em resumo, o modelo com stemming apresentou resultados equilibrados, com bom desempenho na classe negativa e desempenho médio na classe positiva.

UNIVERSIDADE FEDERAL DO CEARÁ

2. Conclusões

*O processo de classificação de textos realizado demonstrou a importância de diferentes etapas no pré-processamento e na escolha das técnicas de vetorização para o sucesso do modelo. A análise incluiu diversas abordagens para transformar e representar os dados textuais, como o uso de **CountVectorizer**, **TF-IDF**, **Word2Vec**, e técnicas de **lemmatização** e **stemming**.*

- ***CountVectorizer e TF-IDF**: Ambas as técnicas mostraram um desempenho satisfatório em termos de precisão, recall e F1-score, com o **TF-IDF** apresentando um ligeiro avanço. Isso é esperado, pois o TF-IDF leva em consideração a frequência relativa de termos e sua importância no contexto, resultando em um modelo que pode discernir melhor a relevância de cada palavra no texto.*
- ***Word2Vec**: Embora oferecesse uma boa captura de relações semânticas entre palavras, a técnica de **Word2Vec** teve um desempenho inferior quando comparada a **CountVectorizer** e **TF-IDF**. A diferença de desempenho pode ser atribuída à maior complexidade e ao fato de Word2Vec exigir uma maior quantidade de dados para aprender representações eficazes de palavras.*
- ***Pré-processamento (Lemmatização e Stemming)**: A escolha entre **lemmatização** e **stemming** também impactou o desempenho dos modelos. Ambas as abordagens melhoraram o desempenho, mas a **lemmatização** geralmente gerou resultados mais robustos, pois preserva o significado e a forma base das palavras.*

*Em termos de desempenho geral, o modelo treinado com **TF-IDF** e **lemmatização** apresentou os melhores resultados, com uma precisão de 76% e uma avaliação equilibrada entre as classes, o que é ideal para tarefas de classificação com desequilíbrio entre as classes.*

Esse estudo ilustra como as diferentes abordagens de pré-processamento e representação de texto podem afetar os resultados de modelos de classificação. Para futuros aprimoramentos, seria interessante explorar outras técnicas de vetorização, ajustar hiperparâmetros dos modelos e realizar validação cruzada para garantir a robustez e a generalização do modelo.

UNIVERSIDADE FEDERAL DO CEARÁ

3. Próximos passos

***Aprimorar Modelos**: Testar outras técnicas de vetorização, como **FastText** ou **BERT**, para comparar o desempenho.*

***Ajustar Hiperparâmetros**: Realizar tuning dos hiperparâmetros dos modelos para melhorar a precisão.*

***Validação Cruzada**: Implementar validação cruzada para avaliar a robustez do modelo e evitar overfitting.*

***Análise de Erros**: Examinar os erros do modelo para identificar padrões e melhorar o pré-processamento ou os algoritmos.*

***Expansão do Conjunto de Dados**: Aumentar o volume de dados de treinamento, se possível, para melhorar a capacidade do modelo de generalizar.*